Title: Red light Green light.

Abstract

Historically baseball games are close in Score with one or two scores can decide the game. The idea behind this project is to find the best pattern for a baserunner to Steal 1st base bring them close to home base.

Data was ~~used~~ collect from SMT and fitered down to ball-events, lineups, & player positiioning which is further taken to just runner 1, secondbaseman, & shortshop. Positioning using Python & Decision trees the model found defensive positing & movement was the biggest indicator if a steal would be successful. Using this model we tested several feature related to runner, secondbaseman, & Shortshtop positioning. If we use runner positioning, velocity & defense distance covered per step our model predicts with 100% accuracy showing that data is leaking & the model knows the answer instead of predicting. We also ~~predicted~~ try predicted is Catcher throw the influenced Steals. & Catcher avg 2.42 sec to seccond baseman & 2.41 to Shortstop with the succes rate of Stealing on Second baseman being 70.00% while its only 62.75% on shortstop

Conclusion..

what origanlly Started with a big idea impractical work turned into a Simple flow Chart. But I will find it funny seeing a coach take out this Giant flow Chart & Try to See if a runners should ~~see~~ steal.

In the future I would like to add in ball positions to see where the ball was Caught & further expand the model to cover Steals for third base and home. Right now I am not Sure if there is a Steal graphic predictor. but Steals can be an important part of a game.

Now forward feature selection does have its faults and one of them is that it used all features in our dataframe. So adding some customation to it by removing certain features that are skewing the model. we used a Decision tree to test our model which is like a flow chart. I basically ran 4 different test removing certain features. The first test I did. I dropped all colums that had to do with the ~~runner~~ base runner. which after testing left us with 4 features, velocity, acceleration, Jerk, & distance covered. being the colums with the highest predicting chance. The test score shows how accurate the model is which is 74% of the time.

~~The next 2 tests~~

The Model

Using machine learning to find a pattern of the best time to steal. Using our dataframe we want to find our y which would be if a steal was successful or not. To find that we use our X which would be our features / other colums to help us predict our y.

In our current dataframe we have 24 feature colums to help us predict our y. Testing these feature is a time consuming process because you have to find different combinations get a score & keep the highest score. This would be over 1 million combination so instead of doing that we will use a techinical called ~~back~~ forward feature selection. that will test all feature for us one at a time then tell us the best top features to use.

Graph.

this is the feature we used for the model. what it says is this many features gets a score and our top score was 4 features.

Plotting our throw times on a Density curve with the ~~stol~~ Steal outcome we ~~can~~ measure the peak density ~~peak~~.. Second base man has around the same peak for both successful & failed Steals being ~~around~~ between 2.4 - 2.5 secs The Shortstop has different density peaks with a failed Steal being between 2.3 - 2.4. secs. from catche to short stop & a Successful Steal being 2.4 - 2.5 Showing that .5 secs can make the difference between a. successful/failed Steal..

Density Graph.

finally we add in our throw time & who caught the ball~~s~~ into our dataframe.. Using this we can now calculate offensive & defensive, change positioning, runner time, distance covered per step, total distance covered, velocity, acceleration, & Jerk time.

to figure out if that was an actual steal I need to make sure its the same batter. by adding in lineups data I can see batter # and using that can find If batter is the same and next play someone moved from 1st to 2nd it was probley a steal. I can further expand on this by ~~if~~ labeling this successful steals then using the same method if batter is the same but next play ~~first~~ runner 1 disapears probely a failed steal.
Now we ~~ha~~ technically have enough data to make a machine learning model or just positioning data but I wanted to see if catcher throw time could effect the play. This is where the ball-event data comes in. by filtering plays in a sequence to pitcher pitches, catcher catches, catcher throws, second baseman or shortstop catches we can figure out the throw time.

Putting that graph into a table it was found that you have a higher SR on second baseman than shortstop. with 35 ~~steals~~ successful steals : 15 failed steals. the shortstop had 32 successful steals : 19 failed ~~steal~~ steals.

Screen will flash alternating between Green & Red before finally stateing Steal Chance 70%. Green light. If Squid games song is copyrighted then a chart will be used instead. Unfortually I personally am not graphically competent enough to make this so I made a simple flow chart. based on our Decision tree model that can help players or coaches or fans if a steal should happen. Fans can stop at the top and be asked questions until a decision happens. which we will address in a later section.

Data!

Getting into the technical side with ~~first~~ the data we used. first data was used for figuring out where players. player-positions are during a play or for us a steal attempt. what I did was get feild positioning on runner 1, runner 2, second baseman & shortstop. then I only need to keep positioning data ~~where~~ when someone is on first base. ~~To~~ I have to add new colums label True or False if firstbase & second base is occupied & I can use that info to figure out when a steal happens ~~but~~ by seeing that when 1st base is occupied but second base isnt the next play 1st base is emply but second base is occupied we can tell that maybe a ~~steal~~ steal happened.

Intro.

According to Team Ranks sports data. their was a total of 4,475 Steals attempted during ~~MLB~~ ~~2023~~ seasons with the 2023 MLB.

~~Teams avg a total of .70 Steals per game.~~

while this sounds like alot of steals. it is actually .70 Steals per game. which when it happens ~~does not~~ Fans do not make a big deal out of it. This problem is what I hoped to solve with my Study how we can make ~~Steals~~ Steals more exciting with graphics ~~or~~ or games increasing Fan engagement. Hopefully this can be used not only during actually Steal attempts but when their is a lull in the game and Someone is on first base ~~to~~ show his ~~a~~ percentage of a successful Steal. I call this Red light Green light Should he go using machine learning, on ~~Player~~ offensive/defensive positioning to Calculate the Successrate (SR) chance of a base runner Stealing Second base. My original idea for this graphic was ~~having the Squid games music playing~~. having the Stadium blacked out then the Squid games red light green light Song will play. ~~Showing a player on~~ ~~[illegible]~~ with a player on 1st base. The Screen will ~~ask~~ highlight ~~the~~ runner 1, second baseman, Shortstop, ~~[illegible]~~ positioning: ~~Catcher throw speed~~ ; then show catcher throw speed It the will ask the crowd Steal chance Green for yes Steal Red for No Steal.